

Alemeno Internship Assignment: Saner

Project summary and implementation approach

1. Problem Understanding

The assignment asks for an Android React Native application that can open a live camera feed, detect only the provided correct markers, reject the provided incorrect markers, and show the processed marker as a tight 300x300 output. The app also needs to gather up to 20 accepted marker captures and remain practical on a real device.

2. High-Level App Flow

- Open the native camera preview using react-native-vision-camera.
- Capture a photo snapshot when the user taps Capture & Analyze.
- Crop the central guide region and tighten it around the darkest marker blob.
- Normalize the candidate crop to 300x300.
- Validate the marker using border and anchor structure rules.
- If accepted, add the processed crop to the in-app accepted-marker gallery.

3. Main Technical Decisions

The implementation is snapshot-based instead of a continuous frame processor. This keeps the flow simpler and easier to reason about while still satisfying the assignment goal of camera-based marker validation on Android.

- The output size is fixed at 300x300 to match the requirement exactly.
- Camera format selection prefers a photo resolution around 2560x2560 and aims to stay within the required 2000-3000 range.
- Marker analysis is split into small scanner modules so the logic is easier to maintain and tune.

4. Marker Detection Logic

The current detector is based on structure rather than object recognition. It looks for a square-like marker with a strong border pattern and one valid corner anchor. The analyzer does not rely on one specific animal image in the center.

- Blob-first crop: the darkest marker-like region is isolated before detailed scoring.
- Ring-based border analysis: an outer ring should be dark while the inner ring just inside it should remain relatively bright.
- Corner-anchor analysis: one corner should dominate as the anchor while the other corners remain comparatively light.
- Oversized-anchor rejection: markers with an excessively large corner block are rejected.
- Center-square rejection: markers that place the dark square in the center instead of a corner are rejected.

5. Scanner File Structure

- app/camera.tsx: camera UI, capture flow, result gallery, and live analysis summaries.
- app/marker-debug.tsx: manual sample browsing for quick sanity checks.
- src/features/scanner/analyzeCapturedMarker.ts: public scanner entry point.
- src/features/scanner/markerAnalyzerCore.ts: blob crop, normalized analysis, and scoring.
- src/features/scanner/markerPixelUtils.ts: low-level pixel sampling helpers.
- src/features/scanner/markerPreview.ts: grayscale debug preview generation.
- src/features/scanner/markerAnalysis.types.ts: shared scanner types and constants.

6. Validation Baseline

During manual testing, the following cases were used as the main reference set for tuning and regression checks:

- Correct dog marker accepted with Borders 4/4 and Dark corners 1.
- Correct pig marker accepted with Borders 4/4 and Dark corners 1.
- Rotated correct marker accepted.
- Incorrect oversized-anchor marker rejected.
- Incorrect center-square marker rejected with no valid anchor corner.

7. Deliverables Status

- Android app source code is present in the repository.
- A release APK was built from the runnable project copy.
- The project is branded as Saner.
- This PDF explains the implementation approach and validation strategy.

8. Known Limitations

- The scanner currently works on captured snapshots rather than real-time frame-by-frame validation.
- The detector is heuristic and tuned against the provided reference markers rather than using a learned model.
- A full perspective warp pipeline could improve robustness further for extreme tilt or lighting conditions.

9. Conclusion

Saner is a practical Android assignment submission that captures markers, normalizes them to the required size, validates them using border-and-anchor structure rules, and stores accepted results inside the app. The codebase was also refactored into smaller scanner files to keep the implementation more human-readable and easier to extend.